# Préconception automatique des réseaux de distribution MT/BT

Documentation technique — architecture logicielle, modèle de données et algorithmes du moteur de planification géospatiale de la Plateforme PLE RDC.

| | |
|---|---|
| CLIENT | Agence Nationale de l'Électrification et des Services Énergétiques en Milieu Rural et Périurbain (ANSER) |
| PROJET | Extension de la Plateforme PLE RDC — Module 2 : préconception automatique du réseau de distribution |
| OBJET | Documentation d'ingénierie logicielle (pipeline, schéma `plan`, algorithmes, exploitation) |
| VERSION | 1.0 — 22 octobre 2026 |
| PÉRIMÈTRE | 11 agglomérations du Kwilu et de la Tshopo — 413 355 ménages desservis |
| STATUT | VALIDÉ — RECETTE INTERNE |

**11**
AGGLOMÉRATIONS

**2 891**
POSTES HTA/BT

**1 756 km**
RÉSEAU MT

**10 837 km**
DESSERTE BT


1 Objet et périmètre
2 Architecture logicielle
3 Modèle de données `plan`
4 Pipeline d'exécution
5 Algorithmes de construction
6 Performance et robustesse
7 Visualisation cartographique
8 Validation, limites et évolutions
9 Annexes techniques

# 1 OBJET ET PÉRIMÈTRE

Le module transforme des données ouvertes (bâtiments et voirie OpenStreetMap, complétées par les données de population de la plateforme) en une **préconception géométrique** des réseaux de distribution électrique : emplacement des postes HTA/BT, tracé du réseau moyenne tension (MT) et desserte basse tension (BT) le long de la voirie. Il produit une géométrie exploitable par un bureau d'études, un métrage et une carte interactive, sans réaliser de calcul électrique.

## 1.1 Questions auxquelles le module répond

- Combien de postes HTA/BT faut-il pour desservir une agglomération, et où les implanter ?
- Quel tracé emprunte le réseau MT depuis le poste source, et quelles sont les jonctions et les départs (feeders) ?
- Quel kilométrage de BT faut-il pour relier chaque bâtiment à son poste, et quels bâtiments exigent un épi ?
- Quelles agglomérations présentent des lacunes de voirie et devront être levées par une étude de terrain ?

## 1.2 Contraintes de conception

*Contraintes structurantes imposées au moteur.*

| Contrainte | Mise en œuvre |
|---|---|
| Écriture limitée au schéma `plan` | Aucune écriture dans les tables applicatives ; le schéma est créé et migré par le moteur. |
| `etl.py` et tables applicatives intacts | Le module lit `batiments_rdc` en lecture seule, filtré par emprise. |
| Aucune dépendance Python supplémentaire | SQLAlchemy, GeoPandas, Shapely, pgRouting ; la carte est du HTML/JS autonome. |
| Jamais de jointure globale sur `batiments_rdc` | 84,9 M de lignes : tout accès est filtré par `ST_Contains` sur l'emprise de l'agglomération. Une jointure globale a déjà provoqué un redémarrage du serveur PostgreSQL. |
| Idempotence | Chaque commande remplace intégralement les objets de l'agglomération traitée ; rejouable sans nettoyage manuel. |

> **CE QUE LE MODULE NE FAIT PAS**
>
> Le moteur ne calcule ni chute de tension, ni foisonnement, ni courant de court-circuit, ni section de câble, ni marché de poteaux ou de transformateurs. Les tensions et le plafond de ménages sont des paramètres de dimensionnement destinés à être repris par les modules électrotechniques (voir la documentation destinée à l'ingénieur électricien).

# 2 ARCHITECTURE LOGICIELLE

Le système est une chaîne de traitement **batch, un processus, sans service applicatif** : une base PostgreSQL/PostGIS/PgRouting sert de source et de magasin de résultats, un exécutable Python orchestre les calculs, deux sorties sont produites — une synthèse tabulaire et une carte autonome.

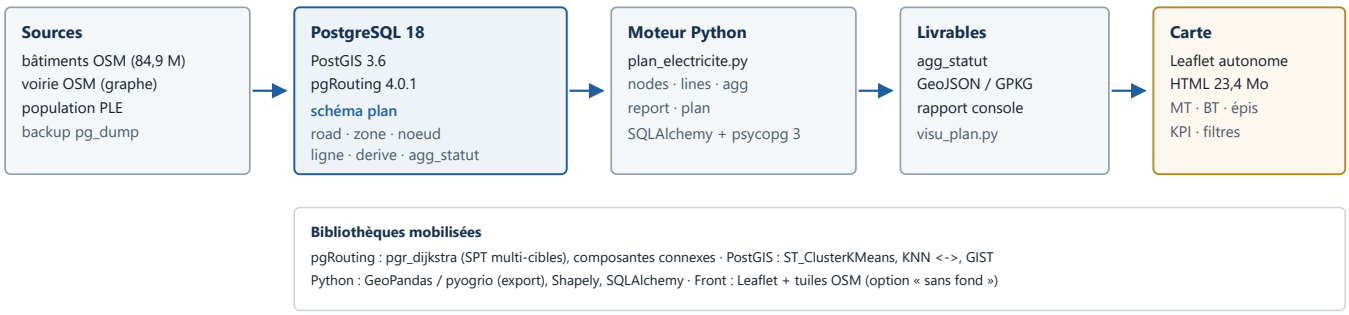



*Figure 1 — Chaîne de traitement de la donnée brute à la carte interactive.*

### 2.1 Composants

| Composant | Rôle | Points de vigilance |
|---|---|---|
| `plan.road*` | Graphe de routage (arêtes, sommets, composantes connexes) | Voirie rurale incomplète : les composantes isolées sont normales. |
| `plan.zone` | Zones de distribution issues de DBSCAN/KMeans | Découpage conditionné par la densité de bâtiments. |
| `plan.noeud` | Poste source, postes HTA/BT, jonctions | Plusieurs postes peuvent partager un sommet routier. |
| `plan.ligne` | Tronçons MT, troncs BT, liaisons directes | Géométrie générique : `LineString` (MT) ou `MultiLineString` (BT). |
| `plan.derive` | Épis BT (bâtiment → tronc) | Un enregistrement par bâtiment raccordé hors voirie. |
| `plan.agg_statut` | Synthèse par agglomération | Recalculée à chaque exécution ; contrôle de cohérence. |

## 3 MODÈLE DE DONNÉES `PLAN`

Le schéma est volontairement plat et dénormalisé : chaque objet porte directement son `agglomeration_id`, ce qui permet de purger et recalculer une agglomération sans toucher aux autres. Les liens topologiques (parent, tronçon, feeder) sont matérialisés par des colonnes, pas par des clés étrangères, pour accélérer les rechargements massifs.

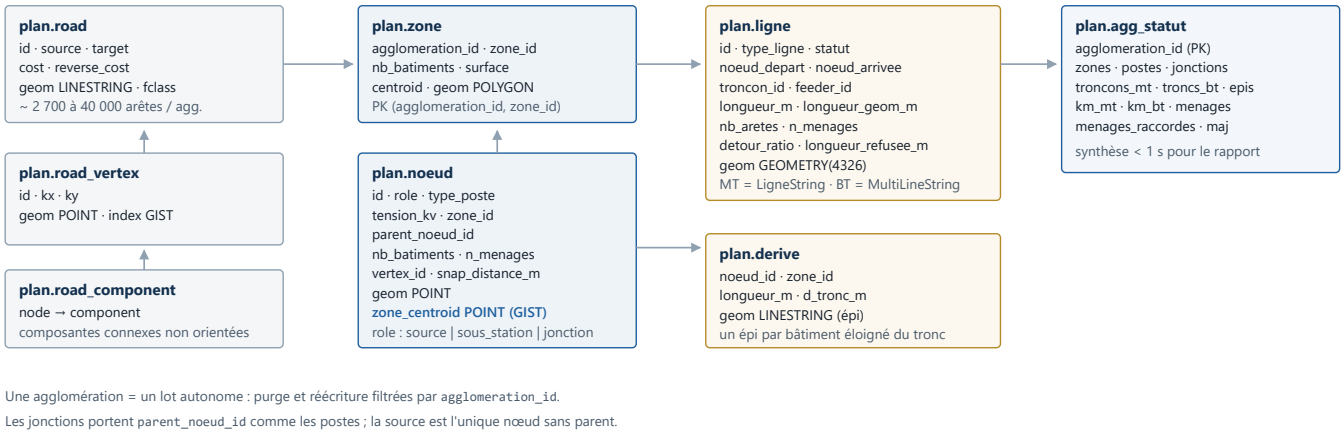



*Figure 2 — Tables du schéma `plan` et relations topologiques.*

### 3.1 Index et intégrité

- `idx_plan_noeud_geom` et `idx_plan_noeud_centroid` (GIST) : KNN des bâtiments respectivement sur le sommet routier et sur le centroïde de zone.

- `idx_plan_road_vertex_geom` (GIST) : accrochage des postes et des bâtiments au réseau.
- `idx_plan_ligne_type` et `idx_plan_derive_agg` : restitution et carte.
- `idx_plan_noeud_role` : filtres source / postes / jonctions dans les requêtes d'arbre.
- Contrôles systématiques : unicité des identifiants de sommets, somme des ménages par agglomération, cohérence `agg_statut` / `ligne`.

## 4 PIPELINE D'EXÉCUTION

Le calcul se fait en deux temps : **nœuds** (postes et affectation des bâtiments), puis **lignes** (MT puis BT). La commande `agg` enchaîne les deux pour une agglomération et `plan` pour un lot complet. Chaque étape est transactionnelle et remplace les objets existants de l'agglomération.

*Commandes principales et effets.*

| Commande | Effet |
|---|---|
| `agg --only 87` | Nœuds puis lignes de l'agglomération 87 si le calcul n'existe pas (sinon ignoré, sauf `--force`). |
| `nodes --only 87` | Poste source, postes HTA/BT, affectation des bâtiments, centroïdes de zone. |
| `lines --only 87` | Tronçons MT, jonctions, troncs BT, épis, statuts et synthèse. |
| `report` | Synthèse console des agglomérations calculées, export tabulaire optionnel. |
| `plan --only 22 32` | Enchaînement complet (périmètre, graphe, zones, nœuds, lignes) pour les identifiants donnés. |

```
export MSYS_NO_PATHCONV=1
cd /c/Users/PC/Documents/Anser_avenat/data

# recalcul d'une agglomération
./venv/bin/python plan_electricite.py nodes --only 32
./venv/bin/python plan_electricite.py lines --only 32

# lot complet, en tâche de fond
for id in 22 32 1087 88 3664 3817 3624 3783 3520 3719; do
  ./venv/bin/python plan_electricite.py nodes --only $id
  ./venv/bin/python plan_electricite.py lines --only $id
done
```

### 4.1 Séquence interne de `lines`

1. Migration de schéma et création des index manquants ( `_migrate_schema` ).
2. Lecture des postes et du poste source, construction du graphe d'arêtes.
3. **MT** : un `pgr_dijkstra` multi-cibles depuis la source, réduction de l'arbre par `lead()` + `DISTINCT ON (child)`.
4. Coupe aux postes et aux jonctions (degré ≥ 3), numérotation `troncon_id`, chaînage des tronçons et affectation des feeders.
5. **BT** : pour chaque poste, accrochage de ses bâtiments à un sommet de sa composante, puis un SPT élagué aux cibles, agrégé en `MultiLineString`.
6. **Épis** : bâtiments au-delà du seuil, distance au tronc via `ST_ClosestPoint`, insertion dans `plan.derive`.
7. Mise à jour de `agg_statut` et journalisation des replis (liaisons directes MT et BT).

# 5 ALGORITHMES DE CONSTRUCTION

### 5.1 Zonage et découpage en postes

Les bâtiments de l'agglomération sont agrégés en zones (DBSCAN, rayon 200 m, minimum 10 points), puis chaque zone trop dense est scindée par KMeans. Chaque zone est ensuite découpée en sous-groupes d'au plus `--max-menages-poste` ménages (200 par défaut). Le découpage utilise un ratio de sécurité **0,85** : les sous-groupes visent 170 ménages afin que l'affectation finale reste proche du plafond. Le centroïde de chaque sous-groupe devient le point d'implantation théorique du poste.

### 5.2 Accrochage des postes

Chaque poste est accroché au sommet routier le plus proche, puis recalé dans la **composante de service** (celle qui porte le plus de postes) si un sommet s'y trouve à moins de `--max-snap-m` (1 500 m). Au-delà, il conserve son propre sommet : le poste sera alimenté par une liaison directe MT. Plusieurs postes peuvent partager un sommet : le réseau MT est mutualisé, mais chaque poste conserve son propre tronc BT.

> **DÉCISION DE CONCEPTION**
>
> Les bâtiments sont affectés au poste via un KNN sur le **centroïde de zone**, pas sur le sommet routier. Ce choix équilibre la charge (moyenne 137–145 ménages, p90 ≈ 180) alors que le KNN géométrique sur la voirie produisait des postes à près de 5 000 ménages en zone rurale, où la voirie est très lâche.

### 5.3 Réseau MT

Un unique appel `pgr_dijkstra(edges, source, {postes})` calcule l'arbre de plus courts chemins. Le résultat est réduit à un arbre par `lead(node) OVER (PARTITION BY end_vid ORDER BY path_seq)` puis `DISTINCT ON (child)`. L'arbre est ensuite coupé aux postes et aux sommets de degré ≥ 3 (jonctions), ce qui produit des tronçons homogènes ; chaque tronçon reçoit un `troncon_id` et un `feeder_id` (premier niveau depuis la source). Les branches sont chaînées par `ST_MakeLine` ordonné, avec extrémités recalées sur la géométrie des sommets.

```
arr  AS (SELECT end_vid, node, edge, agg_cost,
                lead(node)     OVER (PARTITION BY end_vid ORDER BY path_seq) AS child,
                lead(agg_cost) OVER (PARTITION BY end_vid ORDER BY path_seq) AS child_cost
         FROM pgr_dijkstra(:edges, :src, CAST(:dst AS bigint[]))),
spt  AS (SELECT DISTINCT ON (child) child, node AS parent, edge
         FROM arr WHERE child IS NOT NULL AND edge >= 0
         ORDER BY child, child_cost)
```

### 5.4 Desserte BT et épis

Pour chaque poste, les bâtiments sont accrochés au sommet de la **composante de leur poste** (table temporaire `_cv` indexée GIST, construite composante par composante). Le SPT `pgr_dijkstra(src → vids)` est élagué aux seuls sommets cibles, puis agrégé en `MultiLineString`. Les bâtiments dont la distance au tronc dépasse `--bt-seuil-epi` (150 m) reçoivent un épi : segment direct du point le plus proche du tronc au bâtiment. Si un poste n'a aucun tronc (tous ses bâtiments sur son sommet), les épis partent directement du poste.

### 5.5 Replis explicites

| Situation | Comportement | Statut |
|---|---|---|
| Poste hors d'atteinte MT | Liaison droite du centroïde du poste vers le nœud le plus proche de l'arbre | `liaison_directe` (MT) |
| Poste sans tronc BT | Desserte intégrale par épis depuis le poste | `liaison_directe` (BT) |
| Bâtiment > 150 m du tronc | Épi individuel dans `plan.derive` | — |
| Tronçon MT > 15 km | Tronçon refusé et signalé ( `longueur_refusee_m` ) | contrôle |

## 6 PERFORMANCE ET ROBUSTESSE

### 6.1 Points chauds

Le coût dominant est le **KNN des bâtiments** et le **SPT BT par poste**. Trois règles ont été validées empiriquement :

- **Filtrer par emprise avant tout accès aux bâtiments** : une jointure globale sur les 84,9 M de lignes sature le serveur (redémarrages observés).
- **KNN par composante sur une table indexée** : un filtre `component = …` appliqué directement sur `road_vertex` force un balayage complet (17 min pour 84 k bâtiments) ; la table `_cv` par composante ramène l'étape à quelques minutes.
- **SPT élagué aux cibles** : ne conserver que les arêtes menant aux sommets des bâtiments, jamais tout le graphe.

### 6.2 Mesures

| Agglomération | Bâtiments | Postes | Nœuds | Lignes |
|---|---|---|---|---|
| 87 — Bulungu | 13 176 | 96 | 45 s | 6–12 min |
| 22 — Kikwit | 83 840 | 582 | 1 min 30 | 3 min |
| 32 — Kisangani | 176 769 | 1 236 | 10 min | 11 min |
| 8 autres | 6 804 – 47 226 | 47 – 329 | 10–40 s | 10–75 s |

### 6.3 Robustesse

- Transactions par agglomération : une erreur n'altère pas les autres lots.
- Reprise idempotente : purge ciblée puis réécriture complète.
- Journalisation systématique des replis (composantes isolées, postes non atteignables).
- Aucune table temporaire persistante : `_bat_zone` , `_bat_poste` , `_bat_vert` , `_cv` vivent le temps de la session.

## 7 VISUALISATION CARTOGRAPHIQUE

`visu_plan.py export` interroge les cinq couches (périmètre, zones, nœuds, lignes, épis) et écrit un **HTML autonome** dans lequel les données GeoJSON sont inlinées. Aucun serveur n'est nécessaire ; l'ouverture se fait par double-clic, et le fond OSM peut être désactivé pour un usage hors ligne.

| Commande | Usage |
|---|---|
| `export` | Génère `visu/plan_kwilu.html` (23,4 Mo pour 11 agglomérations). |
| `serve --port 8000` | Export puis service HTTP local ( `localhost:8000` ). |
| `check` | Valide le JSON inline, compte les entités, vérifie l'absence de dépendance CDN. |




La couche BT est simplifiée à l'export (`ST_SimplifyPreserveTopology(geom, 0.00002)`, soit ≈ 2 m, et GeoJSON à 5 décimales) afin de rester sous 25 Mo sans dégradation visible à l'échelle d'une agglomération. Le panneau latéral présente les KPI par agglomération et permet de filtrer les couches.

## 8 VALIDATION, LIMITES ET ÉVOLUTIONS

### 8.1 Contrôles automatiques passés

| Contrôle | Résultat (22/10/2026) |
|---|---|
| Somme des ménages = nombre de bâtiments | Exact sur 87, 22, 32, 88 ; cohérent partout via `agg_statut`. |
| Charge par poste | Moyenne 137–145, p90 ≈ 180, maximum 372 (< 2× le plafond). |
| Arbre MT | 11/11 agglomérations : sommets géométriques = tronçons routables + 1. |
| Doublons d'identifiants | Aucun dans `road`, `road_vertex`, `road_component`. |
| Carte | JSON valide, 5 765 lignes, 64 047 épis, aucune dépendance CDN. |

### 8.2 Limites assumées

- **Voirie OSM incomplète** : en zone rurale, les bâtiments s'accrochent au même sommet que leur poste ; la desserte passe alors entièrement par épis, et les postes sur composantes isolées reçoivent une liaison MT directe.
- **Troncs BT non mutualisés** : chaque poste tire son propre arbre ; deux postes voisins peuvent emprunter les mêmes rues, ce qui additionne des tronçons partagés dans le kilométrage.
- **Pas de calcul électrique** : sections, chutes de tension, foisonnement et coûts sont hors périmètre du module.

### 8.3 Évolutions recommandées

1. Tronc BT commun par feeder (arbre couvrant partagé) pour réduire le kilométrage.
2. Ajout d'un module de dimensionnement (sections, chutes de tension, poteaux).
3. API REST et file de tâches pour l'exécution à l'échelle provinciale.
4. Versionnement des jeux de données et des paramètres par scénario.

# 9 ANNEXES TECHNIQUES

## 9.1 Extrait DDL — `plan.noeud`

```
CREATE TABLE plan.noeud (
    id              bigserial PRIMARY KEY,
    agglomeration_id integer NOT NULL,
    role            varchar(20),   -- source | sous_station | jonction
    type_poste      varchar(20),   -- source | poste_hta
    tension_kv      double precision,
    zone_id         integer,
    parent_noeud_id integer,       -- nœud amont (NULL pour la source)
    nb_batiments    integer,
    n_menages       integer,
    surface_batie_m2 double precision,
    vertex_id       bigint,        -- sommet routier d'accrochage
    snap_distance_m double precision,
    geom            geometry(Point,4326),
    zone_centroid   geometry(Point,4326)
);
CREATE INDEX idx_plan_noeud_centroid ON plan.noeud USING gist (zone_centroid);
```

## 9.2 Extraits de code et de paramètres

| Élément | Valeur / localisation |
|---|---|
| Plafond par poste | `--max-menages-poste 200`, ratio `POSTE_SPLIT_RATIO = 0.85` |
| Accroche MT | `--max-snap-m 1500`, `--max-troncon-km 15` |
| BT | `--bt-seuil-epi 150`, tension `0.4 kV` |
| Zonage | `--eps 200`, `--minpoints 10` |
| Fonctions clés | `_compute_nodes_agg`, `_compute_lines_agg`, `_bat_vert`, `_migrate_schema` |
| Schéma | `plan` — aucune écriture hors de ce schéma |

## 9.3 Journal type (extrait)

```
Agglomération 32 (Kisangani) : 1236 postes, 1176 tronçons MT (723.7 km),
    1236 troncs BT (2994.2 km), 26307 épis, 176769 ménages desservis
Agglomération 32 : 125 liaisons MT directes (postes non atteignables par la voirie).
Agglomération 32 : troncs BT routés 1108, directs 128.
RÉUSSITE : 4186 lignes écrites dans plan.ligne.
```

Document établi le 22 octobre 2026 à partir de la version du moteur validée sur les 11 agglomérations. Les valeurs chiffrées sont celles de la recette interne ( `plan.agg_statut` ).